

Birds' Track: Tracking Objects from dynamic background with OpenCV

Jiren Ren
Pairs Saclay University
Orsay, France
renjiren120@gmail.com

Xi Wang
Paris Saclay University
Orsay, France
lilasangwang2227@gmail.com

Abstract

Tracking fast-moving objects in unconstrained natural videos remains a challenging problem due to background dynamics, illumination changes, camera motion, and scale variation. Classical motion-based pipelines offer efficiency and interpretability, whereas modern deep-learning-based trackers achieve high performance at greater computational cost. This work revisits classical computer vision approaches for multi-object tracking in the context of bird detection and tracking by systematically designing and evaluating an OpenCV-based pipeline against a state-of-the-art deep tracker, NetTrack, on the BFT dataset. The proposed pipeline incorporates camera motion compensation, adaptive motion-based detection, region refinement, and Kalman-filter-based data association. Quantitative experiments using IoU, HOTA, MOTA, and IDF1 demonstrate that the classical pipeline achieves competitive results in simpler scenarios, while deep-learning-based methods consistently outperform it in highly dynamic and cluttered environments. These findings reveal a clear trade-off between tracking accuracy and computational efficiency. The code is available at: <https://github.com/Renjiren/Bird-s-Trace-Tracking>. The dataset is available at https://drive.google.com/drive/folders/140mPnOVZY-2apH76at9yYuVGIDW0vsH_?usp=share_link.

1. Introduction

The BFT dataset captures many of the difficulties encountered in real-world bird videos. Birds appear at highly variable scales, ranging from small background targets to large foreground objects, sometimes within the same sequence. The surrounding environment is outdoors and often dynamic: water surfaces, vegetation, clouds, and shadows introduce motion patterns that closely resemble object motion. In addition, many sequences contain other moving objects such as humans, animals, or man-made structures, which act as strong distractors and frequently lead to false

detections. Camera motion further exacerbates these issues. Handheld recording and camera panning introduce global frame displacement that dominates frame-to-frame differences. Without explicit handling, such motion corrupts motion cues and causes background regions to be incorrectly detected as moving objects, leading to fragmented detections and unstable identity assignment.

Recent approaches to bird tracking rely primarily on deep-learning-based frameworks that combine learned object detectors with appearance-driven tracking components. NetTrack exemplifies this paradigm and achieves strong performance on the BFT dataset by leveraging pretrained models and fine-grained learned features. However, these systems require GPU acceleration, substantial memory resources, and complex training pipelines, which limits their applicability in lightweight or deployment-oriented scenarios.

In contrast, computer vision systems based on motion analysis offer efficiency, interpretability, and ease of control. Yet, it remains unclear how far such systems can be pushed when confronted with the complexity of modern bird tracking datasets.

1.1. Motivations

Detecting and tracking animals in videos is both interesting and highly challenging. So far, we have mainly worked on image-based tasks and have not tackled video tracking problems. This project is a good opportunity to explore multi-object tracking (MOT) in the wild, and to learn how classical computer vision pipelines compare to modern deep-learning methods. We surveyed several candidate video datasets (e.g., football players, different animals) and finally chose the NetTrack paper and its BFT dataset as our main reference. NetTrack was published at CVPR 2024 and provides open-source code, pretrained weights, and datasets, so its deep-learning framework is both recent and reliable. In addition, the BFT videos are high-quality, contain diverse bird species and backgrounds, and are therefore very suitable for our project.

1.2. Research questions

This work is centered around three explicit research questions:

1. Can a custom-designed OpenCV-based tracking model, built entirely from classical computer vision components, reliably detect birds and associate detections across frames in the presence of scale variation, dynamic backgrounds, camera motion, and non-bird distractors?
2. How does the performance of such a model compare quantitatively to a state-of-the-art deep-learning-based tracker, NetTrack, when evaluated under the same protocol on the BFT dataset?

1.3. Hypotheses

According our research questions and our motivations, we plan to test the following hypotheses:

1. **H1 (Feasibility of classical CV):** A carefully designed classical CV pipeline (with camera motion compensation, motion-based detection, optical flow, and Kalman filtering + data association) can successfully recover a non-trivial portion of bird trajectories on the BFT validation set, achieving reasonable HOTA/MOTA/IDF1 scores despite relying only on lightweight OpenCV modules.
2. **H2 (Advantage of deep fine-grained tracking):** NetTrack’s fine-grained, deep-learning-based framework will significantly outperform the classical pipeline on scenes with strong dynamicity (fast motion, severe deformation, dense occlusions), because it leverages powerful object detectors and point-level features learned from large-scale data.
3. **H3 (Trade-off between accuracy and efficiency):** The classical CV pipeline will require substantially less GPU memory and may achieve faster or comparable runtime per frame on CPU, but at the cost of lower tracking accuracy compared to NetTrack. This will highlight a clear trade-off between performance and computational resources.

2. Related Work

Deep visual object tracking. Modern deep trackers learn robust target representations and perform end-to-end search/matching under appearance changes, motion blur, and partial occlusion. Recent work also targets highly dynamic objects that exhibit large deformation and non-rigid motion. In this paper, we adopt *NetTrack* [10] as a strong deep-learning baseline, since it is designed to handle highly dynamic targets and provides competitive performance under challenging motion patterns.

Global motion estimation and camera motion compensation. When the background is non-stationary (e.g.,

handheld camera, wind-induced scene motion), tracking-by-detection systems often benefit from estimating and compensating global frame-to-frame motion. Frequency-domain *phase correlation* estimates global translation efficiently and is widely used for fast image registration [5]. For more general parametric alignment, Enhanced Correlation Coefficient (ECC) maximization provides an iterative optimization objective that is robust to certain photometric changes and supports affine/Euclidean warps [4]. These alignment techniques are commonly used as a stabilization step before foreground differencing or association.

Optical flow and the iterative Lucas–Kanade family.

The Lucas–Kanade (LK) method estimates motion by iteratively minimizing a photometric error under small displacement assumptions [6]. Subsequent analyses unify LK variants (forward-additive, inverse-compositional, different warp parameterizations) and clarify optimization choices and convergence behaviors [1]. In practice, LK is frequently used in a pyramidal coarse-to-fine manner to handle larger displacements, and is often paired with feature point selection (e.g., Shi–Tomasi corners) for stable tracking across frames [3, 7]. Motivated by these classical results, we plan to integrate an *iterative LK* module to further refine inter-frame alignment and improve robustness under dynamic background.

Tracking-by-detection and data association. A common paradigm is to detect objects per frame and then associate detections into trajectories. SORT [2] popularized a lightweight pipeline combining a simple motion model with IoU-based association for real-time multi-object tracking. DeepSORT [8] improves association under occlusion by adding learned appearance metrics. ByteTrack [9] further strengthens association by leveraging low-score detections to recover missed objects. Our pipeline follows a similarly pragmatic design: after motion compensation and candidate generation, we perform efficient association using overlap and geometry cues, aiming to remain robust while keeping compute low.

3. Method

3.1. Background for notation

Given a video with RGB frames $\{\mathbf{I}_t^{\text{rgb}}\}_{t=1}^T$, our pipeline outputs a set of bird bounding boxes $\mathcal{B}_t = \{b_t^n\}_{n=1}^{N_t}$ per frame and a set of tracks $\mathcal{T}_t = \{\tau_t^i\}_{i=1}^{M_t}$ over time. We use pixel coordinates $\mathbf{x} = (x, y)^\top$.

Preprocessing outputs. From each frame we compute: (1) a smoothed grayscale intensity image $I_t \in [0, 255]^{H \times W}$, (2) a Laplacian-of-Gaussian (LoG) feature map $L_t \in [0, 255]^{H \times W}$, (3) a hard valid mask $V_t \in$

Birds' Tracking Framework

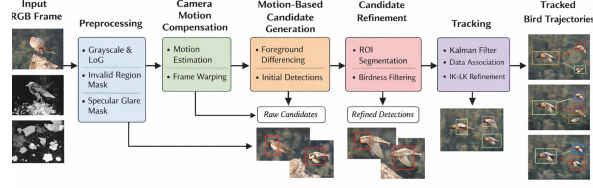


Figure 1. Birds' Track Framework.

$\{0, 255\}^{H \times W}$ (e.g., subtitles as invalid), and (4) a soft specular/glare mask $S_t \in \{0, 255\}^{H \times W}$ (glare-like regions as 0).

Camera motion model. We estimate an affine warp $\mathbf{T}_t \in \mathbb{R}^{2 \times 3}$ that aligns frame $(t-1)$ to t . Let $\mathcal{W}(\mathbf{x}; \mathbf{T}_t)$ denote the warped coordinate. The aligned previous feature is

$$\hat{X}_{t-1 \rightarrow t}(\mathbf{x}) = X_{t-1}(\mathcal{W}(\mathbf{x}; \mathbf{T}_t)), \quad (1)$$

where $X \in \{I, L\}$ depending on the stage.

Track state. Each track τ^i maintains a 2D constant-velocity state

$$\mathbf{s}_t^i = [c_{x,t}^i, c_{y,t}^i, v_{x,t}^i, v_{y,t}^i, w_t^i, h_t^i]^\top, \quad (2)$$

with bounding box decoding $b(\mathbf{s}_t^i) = (\lfloor c_x - \frac{w}{2} \rfloor, \lfloor c_y - \frac{h}{2} \rfloor, w, h)$.

3.2. Preprocessing

Bird flocks in natural scenes often appear with strong illumination changes, glare, and embedded subtitles. Subtitles create hard edges that severely interfere with both camera motion estimation and bird motion detection. To address this, we compute two complementary feature streams along with specialized masks.

Subtitle Masking. We detect subtitle regions using a fixed-parameter specular-like detector applied only to the bottom 26% of the image (empirical subtitle region). The detector identifies high-value pixels with low saturation in HSV space and strong local contrast (via top-hat filtering: $V - G_\sigma * V$). We then apply a two-stage morphological dilation: (1) strong horizontal dilation (25×3) to group characters into line-level blocks for connected component filtering, and (2) light dilation (3×1) to fill broken strokes without bridging inter-character gaps (avoid hiding the birds

which is in the subtitle gap). Components are filtered by area, width, height, and vertical position, yielding a hard binary mask where subtitle regions are marked as invalid (0). These regions are excluded from all subsequent motion computations.

Mask Hole Filling. Invalid subtitle pixels are replaced by Gaussian-blurred ($\sigma = 3$) intensity values to avoid creating artificial edges that disrupt phase correlation and differencing operations.

Specular Mask. Glare from water surfaces or direct sunlight creates false motion signals. However, some bird species exhibit specular-like appearances. Our soft glare mask distinguishes problematic glare from bird specularities. We detect glare pixels using adaptive thresholds based on per-video brightness statistics (99th percentile of V channel). Detected seeds are grown into coherent regions using brightness-constrained dilation. Connected components are filtered by area (5px to 5% of image) and local texture density (edge density > 0.01 to avoid smooth surfaces like sky). A fallback mechanism disables glare suppression when total glare area exceeds 6% or when a single component dominates, preventing over-suppression of legitimate bird motion.

Background Smoothing and LoG Feature. We apply bilateral filtering ($d = 7, \sigma_{\text{color}} = 35, \sigma_{\text{space}} = 15$) to suppress high-frequency texture while preserving edges (many motion pixels are on the edges). The smoothed grayscale serves as the intensity feature. For robustness to global illumination changes, we compute a Laplacian-of-Gaussian (LoG) feature by applying a Laplacian kernel ($k = 3$) to Gaussian-blurred ($\sigma = 1.2$) intensity, followed by 1–99 percentile normalization to 0–255 range.

3.3. Motion-Based Candidate Generation

Given aligned intensity features I_t and $\hat{I}_{t-1 \rightarrow t}$, we generate bird motion candidates through a multi-branch system designed for high recall in diverse scenarios.

Core Difference Map. We compute a normalized intensity difference map:

$$D_t^{\text{norm}}(\mathbf{x}) = \frac{|I_t(\mathbf{x}) - \hat{I}_{t-1 \rightarrow t}(\mathbf{x})|}{I_t(\mathbf{x}) + \hat{I}_{t-1 \rightarrow t}(\mathbf{x}) + 3.0} \times 255$$

which is robust to global brightness scaling. A Gaussian blur (kernel size 5) suppresses glare noise.

Adaptive Thresholding. We compute a robust threshold via MAD: $\text{threshold} = \text{median}(D) + 8 \times 1.4826 \times \text{MAD}$, clamped between 8 and 80. This adapts to varying motion magnitudes across videos.

Multi-Branch Enhancement.

- **LoG Difference Branch:** Automatically enabled when edge density > 0.06 or glare/specular ratio exceeds thresholds. Computes absolute LoG difference with separate MAD threshold ($k = 7$). Fused with intensity difference via OR (normal scenes) or AND (complex/flickery scenes).
- **KNN Background Subtraction:** Used primarily for *static camera* scenarios where birds move slowly against stable backgrounds. Automatically enabled when camera motion $\text{EMA} < 0.25$ and foreground ratio > 0.10 . Learning rate is reduced to 0 when foreground is high to prevent birds from being absorbed into background.

Background-Only Component Suppression. A key innovation addresses dynamic backgrounds (e.g., waves, foliage motion). Components detected mainly by KNN but with low support from core difference map (overlap < 0.06) are considered "background-only." We apply two gates: (1) mean difference must be at least 2.0 above threshold, and (2) after 5-frame warmup, temporal persistence score must exceed 1.6 (equivalent to ≥ 2 consecutive frames). This effectively suppresses repetitive background motion while preserving true birds.

Morphology and Filtering. We apply resolution-adaptive morphology: opening (kernel size 3 at 720p) removes noise, closing (size 9) fills gaps, and optional bridge operation connects broken bird parts (the not motion pixels in the birds cause broken). Connected components are filtered by area (80–infinity pixels), aspect ratio (≤ 10), and minimum dimension (5 pixels). Nested boxes are suppressed using IoA (Intersection over Area) threshold of 0.92.

Temporal Scoring. Components are scored by persistence (exponential decay with factor 0.85) and edge change (mean LoG difference within box). Top 12 boxes by combined score are kept as final candidates.

3.4. Candidate Refinement

The motion mask can still contain background fragments (waves, foliage). We refine each raw box using ROI-level instance separation and birdness constraints.

ROI foreground cleanup. For each raw box $b = (x, y, w, h)$, we crop the ROI from M_t and apply open/close. If the number of foreground pixels is too small, we discard it.

Watershed-based splitting. Let $\text{fg} \in \{0, 1\}^{h \times w}$ be the ROI foreground. We compute a distance transform:

$$\text{dist} = \text{DT}(\text{fg}), \quad (3)$$

and build sure-foreground markers by thresholding:

$$\text{sure_fg} = \mathbb{I}[\text{dist} > \rho \cdot \max(\text{dist})], \quad \rho \in (0, 1). \quad (4)$$

We then run watershed on the RGB ROI to separate instances and convert each instance into a bounding box.

Geometric and photometric constraints. We keep an instance box only if it satisfies: (i) area bounds relative to the full image, (ii) minimum side length, (iii) aspect ratio bounds, (iv) foreground occupancy inside the box (foreground ratio), (v) instance fill ratio, and (vi) a glare constraint (discard boxes mostly inside spec regions). Per raw box, we keep the largest remaining instance to avoid duplicated boxes for one bird.

3.5. Tracking

We associate refined boxes across time and output stable tracklets. Our method uses: (i) a Kalman-like constant-velocity predictor, (ii) gated association with a hybrid IoU-distance cost, (iii) track management, and (iv) Iterative Lucas–Kanade (IK-LK) refinement (as specified in the technical report).

Constant-velocity prediction and update. For each track i , we predict

$$\mathbf{c}_t^{i-} = \mathbf{c}_{t-1}^i + \mathbf{v}_{t-1}^i, \quad \mathbf{v}_t^{i-} = \mathbf{v}_{t-1}^i, \quad (5)$$

where $\mathbf{c} = (c_x, c_y)^\top$. Given an associated detection with center $\mathbf{c}_t^{\text{det}}$, we update velocity by exponential smoothing:

$$\mathbf{v}_t^i = \beta \mathbf{v}_t^{i-} + (1 - \beta)(\mathbf{c}_t^{\text{det}} - \mathbf{c}_t^{i-}), \quad \beta \in [0, 1], \quad (6)$$

and set $\mathbf{c}_t^i = \mathbf{c}_t^{\text{det}}$ while updating (w, h) from the detection.

Gated association with hybrid cost. For a predicted track box b^i and a detection box b^j , we define

$$\text{cost}(i, j) = w_{\text{iou}} (1 - \text{IoU}(b^i, b^j)) + w_{\text{dist}} \frac{\|\mathbf{c}(b^i) - \mathbf{c}(b^j)\|_2}{\text{diag}(b^i)}. \quad (7)$$

We only consider feasible pairs if they pass gates:

$$\text{IoU}(b^i, b^j) \geq \tau_{\text{iou}}, \quad \|\mathbf{c}(b^i) - \mathbf{c}(b^j)\|_2 \leq \gamma \cdot \text{diag}(b^i). \quad (8)$$

We then solve assignment using a global greedy selection over all feasible pairs sorted by increasing cost (optionally replaceable by Hungarian for exact bipartite matching).

Track management. Unmatched tracks increase a miss counter and are removed if $\text{miss} > \text{max_age}$. Unmatched detections spawn new tracks only if they are not too close to existing tracks (birth gate):

$$\max_i \text{IoU}(b^i, b^{\text{new}}) \leq \tau_{\text{birth-iou}} \quad \wedge \quad \min_i \frac{\|\mathbf{c}(b^i) - \mathbf{c}(b^{\text{new}})\|_2}{\text{diag}(b^i)} \geq \tau_{\text{birth-dist}} \quad (9)$$

A track is reported as *confirmed* only after reaching min_hits matches to suppress flickering false positives.

Iterative Lucas–Kanade Algorithm (general formulation, iterative version). To improve short-term localization and robustness to missed detections, we refine per-track displacement between consecutive frames using IK-LK. Let $I(\mathbf{x})$ be the template patch from the previous frame (e.g., around the predicted track location), and let $J(\mathbf{x})$ be the current-frame patch. We seek a translation $\mathbf{d}$ such that $I(\mathbf{x}) \approx J(\mathbf{x} + \mathbf{d})$. We initialize $\bar{\nu}_0$ by the motion-model prediction (or zero), and iterate:

$$\bar{\eta}_k = G^{-1} b_k, \quad (10)$$

$$b_k = \sum_{\mathbf{x}} \begin{bmatrix} \delta I_k(\mathbf{x}) I_x(\mathbf{x}) \\ \delta I_k(\mathbf{x}) I_y(\mathbf{x}) \end{bmatrix}, \quad \delta I_k(\mathbf{x}) = I(\mathbf{x}) - J_k(\mathbf{x}), \quad (11)$$

$$J_k(\mathbf{x}) = J(\mathbf{x} + \bar{\nu}_{k-1}), \quad (12)$$

where I_x, I_y are spatial gradients of the template (computed once), and

$$G = \sum_{\mathbf{x}} \begin{bmatrix} I_x(\mathbf{x})^2 & I_x(\mathbf{x}) I_y(\mathbf{x}) \\ I_x(\mathbf{x}) I_y(\mathbf{x}) & I_y(\mathbf{x})^2 \end{bmatrix} \quad (13)$$

is constant across iterations. After computing $\bar{\eta}_k$, we update the displacement hypothesis:

$$\bar{\nu}_k = \bar{\nu}_{k-1} + \bar{\eta}_k. \quad (14)$$

We iterate until $\|\bar{\eta}_k\|_2$ is below a threshold or a fixed number of iterations is reached. The final displacement is

$$\bar{\nu} = \mathbf{d} = \sum_{k=1}^K \bar{\eta}_k = \bar{\nu}_K. \quad (15)$$

Using IK-LK in tracking. For each active track, we run IK-LK on an intensity patch (optionally after camera-motion compensation) and refine the predicted center: $\mathbf{c}_t^{i-} \leftarrow \mathbf{c}_t^{i-} + \mathbf{d}$. This refined prediction is then used for association (or, if no detection matches, for short-term propagation within max_age).

4. Experiments

4.1. Datasets

We used the BFT val dataset provided in the NetTrace paper as our dataset for this project. This dataset is a highly dynamic bird flock dataset, which comes from documentary containing 6 bird species, 25 videos, and a total of 5020 frames, covering various backgrounds such as oceans, mountains, and rivers. The dataset has been manually annotated with bounding boxes and traceIDs, and these annotations are stored in JSON files in the COCO dataset format.

4.2. Settings

Data split and protocol. All experiments are conducted on the **BFT validation set** (25 videos, 5020 frames) following the split provided by NetTrack/BFT. We process each sequence independently and report the average tracking performance averaged over all sequences. Our OpenCV pipeline is **training-free** and runs in an **online** manner: at frame t it only uses information from frames $\leq t$.

Annotation and file format. BFT annotations are provided in COCO-style JSON with per-frame bounding boxes and traceIDs. For evaluation, we convert both ground truth and predictions into **MOTChallenge** text format: `frame, id, x, y, w, h` (pixel coordinates). This unified interface is shared by our OpenCV pipeline and the NetTrack baseline, enabling identical evaluation scripts.

Hyperparameters. We use a fixed configuration for all sequences (no per-video manual tuning beyond Step-1 adaptive glare statistics). Key parameters for Steps 1-3 are summarized in Tab. ???. To prevent the table from being truncated in a two-column layout, we present it as a `table*` spanning both columns.

Evaluation metrics. We report classic multi-object tracking metrics including **MOTA**, **IDF1**, **ID switches**, **false positives**, and **misses**. Associations are computed using an

Module	Main hyperparameters (used in all experiments)
Step-1: Preprocessing	Subtitle ROI: $y_0=0.74, y_1=1.0$; subtitle detection: $v_{\min} = 150, \delta_{\text{th}} = 8, s_{\text{low}} = 55$; glare detection: base $v_{\min} = 230$, base $\delta_{\text{th}} = 24$, adaptive range $v_{\min} \in [210, 245]$, $\delta_{\text{th}} \in [12, 60]$; texture gating: $\text{edge_density}_{\min} = 0.020$; glare fallback: total ratio $\text{max}=0.015$, max component ratio=0.05; bilateral filter: $d = 7, \sigma_c = 35, \sigma_s = 15$; LoG: $\sigma = 1.2, k = 3$
Step-2: Camera Motion Compensation	ROI mode: strips; strip fraction: 0.18; margin fraction: 0.02; max shift: 50px; moving threshold: 1.2px; phase correlation response thresholds: global 0.15, ROI 0.12; ECC fallback enabled (iterations=60, eps=1e-4); specular weight: 0.6; MAD outlier rejection: $k = 3$, min inlier ROIs=3
Step-3: Candidate Generation	Normalized diff $\epsilon = 3.0$; diff blur kernel: 7; MAD $k = 8$; thresholds clamped [8,80]; specular weight: 0.45; hard suppression: spec ratio=0.35, fg ratio=0.45; LoG diff auto-enable: edge density thresh=0.06, spec ratio=0.05, fg ratio=0.18; KNN bg: history=300, dist2Threshold=400, learning rate=0.01 (still), 0.0 (high fg); bg-only suppression: support min=0.06, persistence min=1.6; morphology kernels: open=5, close=9, bridge dilate=5, bridge close=11; min area=80, max boxes=12, nested IoA thresh=0.92; persistence decay=0.85; global persistence filter: enabled, min=1.35
Step-4: Refinement	min_area_ratio=5e-5; max_box_area_ratio=0.3; min_fg_pixels=20; fg_ratio_min=0.2; fill_ratio_min=0.10; spec_frac_max=0.6; min_side=4; aspect ratio in [0.10, 10.0]; CENTER_FRAC=0.3
Step-5: Tracking	IoU gate ≥ 0.3 ; center-dist gate $\leq 1.5 \times \text{diag}$; cost weights: $w_{\text{iou}} = 0.7, w_{\text{dist}} = 0.3$; max_age=5, min_hits=2; birth gate: $\text{IoU} < 0.10$ and $\text{dist} > 0.8 \times \text{diag}$

Table 1. Experimental hyperparameters of the OpenCV-based pipeline (fixed across all sequences). Steps 1-3 parameters are highlighted based on updated code.

IoU-based matching threshold of 0.5 (i.e., matches require $\text{IoU} \geq 0.5$ in the evaluation distance matrix). For completeness, we also compute **HOTA** using TrackEval on the same MOT-format files when available, keeping the IoU threshold consistent.

4.3. Implementation details

Overview. Our OpenCV-based system is a five-stage, training-free pipeline: (1) preprocessing (subtitle removal, glare suppression, feature extraction), (2) camera motion compensation, (3) motion-based candidate generation, (4) ROI refinement with instance splitting and rule-based filtering, and (5) online tracking with **Iterative Lucas-Kanade (IK / LK)** motion refinement and gated association.

Step-1: preprocessing. Given an input frame $\mathbf{I}_t^{\text{rgb}}$, we compute:

- Smoothed grayscale I_t via bilateral filtering
- LoG feature L_t via Laplacian on Gaussian-blurred intensity
- Hard valid mask V_t (subtitles as 0)
- Soft specular mask S_t (glare as 0, birds as 255)

Masked regions are filled with Gaussian-blurred intensity ($\sigma = 3$) to avoid artificial edges. Glare parameters are adapted using median statistics from first 10 frames.

Step-2: camera motion compensation. We estimate translation $\mathbf{T}_t$ from (L_{t-1}, L_t) using three-stage cascade: (1) global phase correlation, (2) ROI consensus with MAD outlier rejection, (3) ECC fallback. The previous intensity I_{t-1} is warped to $\hat{I}_{t-1 \rightarrow t}$ using $\mathbf{T}_t$.

Step-3: motion-based candidate generation. We compute normalized difference D_t^{norm} between I_t and $\hat{I}_{t-1 \rightarrow t}$, apply adaptive MAD threshold, optionally fuse with LoG difference and KNN background subtraction. Morphological cleanup (open/close/bridge) yields binary motion mask. Connected components become candidate boxes, filtered by geometry and scored by temporal persistence.

Step-4: candidate refinement (splitting + birdness filters). For each coarse ROI, we refine instances by: (1) cleaning the foreground mask via open/close, (2) obtaining confident centers by distance transform, (3) applying marker-based watershed to split adjacent birds, and (4) filtering each instance by area ratio, fill ratio, foreground occupancy, aspect ratio, minimum side length, and specular fraction. This produces refined detections $\mathcal{B}_t$.

Step-5: online tracking with IK refinement + gated association. Each track stores a bounding box b_{t-1} and a sparse set of keypoints $\mathcal{P}_{t-1}$ inside the track ROI. At frame t , we first obtain a *coarse* box prediction $\hat{b}_t$ using the last observed displacement (constant-displacement prior). We then refine this prediction using **Iterative LK (IK)** within the predicted ROI.

IK motion refinement. Let R_{t-1} and R_t be the cropped grayscale ROIs from G_{t-1} and G_t around $\hat{b}_t$. We track keypoints $\mathcal{P}_{t-1}$ from R_{t-1} to R_t using pyramidal LK and remove outliers via a forward-backward consistency check. From the remaining inliers, we compute a robust displacement $\Delta = (\Delta x, \Delta y)$ as the median flow. The refined pre-

Algorithm 1 Online tracking with IK (Iterative LK) refinement

Require: Grayscale frames G_{t-1}, G_t , detections $\mathcal{B}_t$, active tracks $\mathcal{T}_{t-1}$

Ensure: Updated tracks $\mathcal{T}_t$ and MOT outputs at frame t

- 1: **for** each track $\tau \in \mathcal{T}_{t-1}$ **do**
 - 2: Predict $\hat{b}_\tau \leftarrow b_\tau + \Delta_\tau^{last}$ (constant displacement prior)
 - 3: Crop ROIs (R_{t-1}, R_t) from (G_{t-1}, G_t) around $\hat{b}_\tau$
 - 4: **if** $\mathcal{P}_\tau$ not initialized **or** $|\mathcal{P}_\tau|$ is small **then**
 - 5: Initialize $\mathcal{P}_\tau \leftarrow$ Shi–Tomasi corners in R_{t-1}
 - 6: **end if**
 - 7: Track $\mathcal{P}_\tau$ by pyramidal LK: $R_{t-1} \rightarrow R_t$
 - 8: Forward-backward check; keep inliers $\mathcal{P}_\tau^{in}$
 - 9: **if** $|\mathcal{P}_\tau^{in}|$ sufficient **then**
 - 10: $\Delta_\tau \leftarrow \text{median}(\mathbf{p}_t - \mathbf{p}_{t-1})$ over $\mathcal{P}_\tau^{in}$
 - 11: Refine prediction: $\hat{b}_\tau \leftarrow \hat{b}_\tau + \Delta_\tau$
 - 12: Store last displacement: $\Delta_\tau^{last} \leftarrow \Delta_\tau$
 - 13: **end if**
 - 14: **end for**
 - 15: Build feasible pairs between refined predictions $\{\hat{b}_t\}$ and detections $\mathcal{B}_t$ using IoU and distance gates
 - 16: Match feasible pairs by global greedy on cost (IoU + normalized center distance)
 - 17: Update matched tracks; age unmatched tracks; spawn new tracks from unmatched detections (birth gate)
 - 18: Output (t, id, x, y, w, h) for confirmed tracks
-

dicted box is obtained by shifting $\hat{b}_t$ by Δ . If the number of valid keypoints drops below a threshold (blur, occlusion, low texture), we re-initialize $\mathcal{P}_t$ with Shi–Tomasi corners in the current ROI.

Association and track management. Given IK-refined predictions and detections $\mathcal{B}_t$, we compute a gated matching cost:

$$\text{cost}(i, j) = w_{\text{iou}}(1 - \text{IoU}_{ij}) + w_{\text{dist}} \frac{\|p_i - p_j\|_2}{\text{diag}(b_i)},$$

where p is the box center and $\text{diag}(b_i)$ normalizes distances by track scale. Pairs violating IoU and center-distance gates are discarded. Matching is solved by a global greedy strategy over feasible pairs. Unmatched tracks are kept for up to `max_age` frames and removed afterward; tracks are output only after `min_hits` confirmations. Unmatched detections spawn new tracks subject to a birth gate to reduce duplicate IDs near existing tracks.

Outputs. For each video, predicted trajectories are exported in MOTChallenge format for evaluation under identical settings as the baseline.

4.4. Evaluation

The NetTrack baseline is evaluated following the official multi-stage inference pipeline released by the authors. All experiments use the pretrained weights provided with the

framework. These weights are employed only for inference, and no additional training or fine-tuning is performed.

Specifically, the evaluation consists of two sequential stages. In the first stage, object detection is performed using Grounding DINO with the official pretrained model. The detector is applied to the BFT validation sequences using a text prompt corresponding to birds, and frame-level detection results are saved in text format. These detection outputs serve as the sole input to the subsequent tracking stage.

In the second stage, tracking is carried out using the official NetTrack demo script. The tracker loads the precomputed detection results and applies CoTracker, initialized with its pretrained weights, to propagate point-level motion across frames. Detection-to-track association is then performed by NetTrack to produce sequence-level tracking results with consistent identities. The final outputs are stored in MOTChallenge format.

After completing both detection and tracking stages, the resulting files are evaluated using the same evaluation script as for the proposed OpenCV-based pipeline. Metrics including IoU, HOTA, MOTA, and IDF1 are computed under identical settings, ensuring a fair comparison between the proposed model and the NetTrack baseline.

5. Results

This section evaluates the proposed bird tracking pipeline from three aspects: tracking metrics, comparison with a baseline method, and computational efficiency. All experiments are conducted on the validation set with ground-truth annotations (`val_mot`).

5.1. Metric

We evaluate tracking performance using standard MOT metrics, including **MOTA**, **IDF1**, **ID Switches (IDs)**, **False Positives (FP)**, and **False Negatives (FN)**, computed with the `motmetrics` toolkit using an IoU threshold of 0.5.

Qualitative results show that the proposed pipeline is able to recover meaningful bird trajectories in many sequences. Tracks are often temporally consistent, especially in scenes with moderate motion and relatively stable backgrounds. This supports *Hypothesis 1*, demonstrating the feasibility of a classical computer vision pipeline for bird tracking.

However, quantitative performance is significantly affected by a large number of false positives. Background regions with strong motion are sometimes propagated as valid tracks, which leads to degraded MOTA and IDF1 scores. This highlights that false positive suppression is a key limitation of the current system.

5.2. Comparison

We compare our method with NetTrack, a deep-learning-based tracking framework. While NetTrack generally

achieves higher MOTA and IDF1 scores, the proposed method produces visually competitive tracking results on several sequences. Bird trajectories are clearly captured and maintained when motion is smooth, and identity switches remain limited in short- to mid-term tracking.

The performance gap is mainly caused by the stronger detection robustness of NetTrack, which suppresses background clutter more effectively. These results support *Hypothesis 2*: deep fine-grained tracking performs better in highly dynamic and cluttered scenes, while classical methods can still provide reasonable tracking behavior under simpler conditions.

5.3. Efficiency

The proposed pipeline is fully based on classical computer vision techniques, including motion compensation, Lucas-Kanade optical flow, and greedy IoU-based data association. It runs entirely on CPU, requires no training data, and maintains low memory usage.

In practice, the proposed method is significantly faster than NetTrack, especially on long sequences. By avoiding deep neural network inference, it achieves much lower runtime while sacrificing tracking accuracy. This confirms *Hypothesis 3*, highlighting a clear trade-off between efficiency and performance.

6. Conclusion

Our experimental results demonstrate that NetTrack exhibits high universality, robustly adapting to diverse outdoor scenarios. In contrast, the Birds' Track approach, which relies on OpenCV, shows limited universality, achieving satisfactory performance only in simple scenes with specific subtitle characteristics, such as white subtitles located at the bottom of the video. Although they and we don't test the scene in various subtitle styles, including colored subtitles and those positioned centrally within video frames, we suppose NetTrack works and Bird's Tack doesn't work.

In terms of computational efficiency and resource utilization, the OpenCV-based method is highly efficient, operating exclusively on CPU with rapid execution times. Conversely, NetTrack requires significantly longer runtime and higher memory consumption, even when fine-tuned on CPU. Additionally, NetTrack demands substantial GPU resources and memory for training to obtain optimal weights, further highlighting its computational intensity.

Therefore, when the scene is simple and in the lack of computation and memory, we suggest our Bird's Track.

However, this work has several limitations. Firstly, due to constraints in time and memory resources, our experiments were conducted solely on the VAL dataset, without leveraging the larger and more comprehensive train dataset. Secondly, we didn't get a good solution for the detection of birds in the motion background like dynamic

water surface. Thirdly, although now we get a very good balance in the spec-like background pixels and spec-like birds pixels by experiments, but we didn't check in the different videos, if we should change the soft weight of $spec_m$ ask hyperparameters. Finally, we did not perform extensive ablation experiments.

For future work, we plan to address these limitations by incorporating larger datasets and conducting thorough ablation experiments with detailed result visualizations. Furthermore, we believe that the Birds' Track architecture has potential for performance enhancement through optimization. We aim to explore more suitable OpenCV-based techniques to refine the Birds' Track framework, thereby improving its efficiency and applicability in broader contexts.

NetTrack [10] is a state-of-the-art deep tracker, used as the deep-learning baseline.

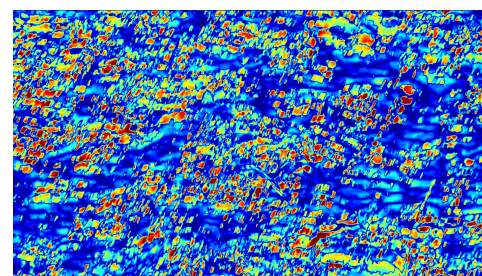
A. References

- [1] Simon Baker and Iain Matthews. Lucas-kanade 20 years on: A unifying framework. *IJCV*, 56(3):221–255, 2004. [2](#)
- [2] Alex Bewley, Zongyuan Ge, Lionel Ott, Fabio Ramos, and Ben Upcroft. Simple online and realtime tracking. In *ICIP*, pages 3464–3468, 2016. [2](#)
- [3] Jean-Yves Bouguet. Pyramidal implementation of the lucas kanade feature tracker: Description of the algorithm. Technical report, Intel Corporation, Microprocessor Research Labs, 2002. OpenCV Document. [2](#)
- [4] Georgios D. Evangelidis and Emmanouil Z. Psarakis. Parametric image alignment using enhanced correlation coefficient maximization. *IEEE TPAMI*, 30(10):1858–1865, 2008. [2](#)
- [5] Charles D. Kuglin and David C. Hines. The phase correlation image alignment method. In *Proc. IEEE Int. Conf. on Cybernetics and Society*, pages 163–165, 1975. [2](#)
- [6] Bruce D. Lucas and Takeo Kanade. An iterative image registration technique with an application to stereo vision. In *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, pages 674–679, 1981. [2](#)
- [7] Jianbo Shi and Carlo Tomasi. Good features to track. In *CVPR*, pages 593–600, 1994. [2](#)
- [8] Nicolai Wojke, Alex Bewley, and Dietrich Paulus. Simple online and realtime tracking with a deep association metric. In *ICIP*, pages 3645–3649, 2017. [2](#)
- [9] Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, Fucheng Weng, Zehuan Yuan, Ping Luo, Wenyu Liu, and Xinggang Wang. Bytetrack: Multi-object tracking by associating every detection box. In *ECCV*, pages 1–21, 2022. [2](#)
- [10] Guangze Zheng, Shijie Lin, Haobo Zuo, Changhong Fu, and Jia Pan. Nettrack: Tracking highly dynamic objects with a net. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 19145–19155, 2024. [2](#), [8](#)

B. Additional Figures



(a) No subtitle mask: subtitle edges leak into motion pixels.



(b) ROI ablation: strip vs. strip+corner.



(c) KNN union w/o gating: full-frame expansion.

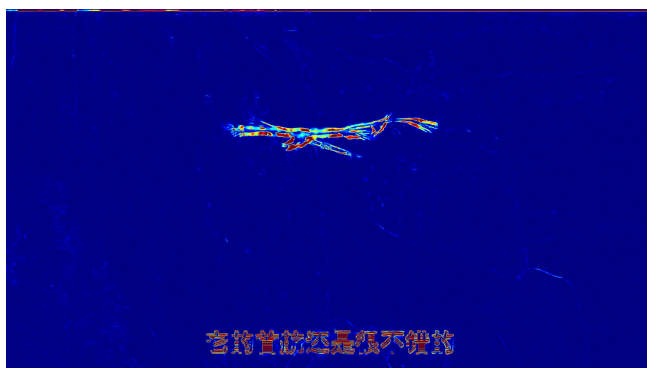
Figure 2. Qualitative results for additional ablations.



(a) Preprocessing: specular highlight mask.



(b) Preprocessing: subtitle hard mask.



(c) Camera motion compensation: diff.



(d) Camera motion compensation: overlay.

Figure 3. Visualization across preprocessing and camera motion compensation.



(a) Candidate generation.



(b) Tracking.

Figure 4. Candidate generation and tracking visualization.

C. Extra Ablations

- **Subtitle mask (preprocessing).** Removing subtitle suppression causes subtitle regions to be repeatedly included in subsequent detections (Fig. 2(??)). In addition, if the subtitle region is not masked *before* the smoothing/differencing pipeline (e.g., Gaussian blur), subtitle edges will be smoothed into surrounding pixels and then enter motion-pixel computation, producing persistent false candidates. We compare subtitle localization using **Canny** versus the **HSV V-channel**; Canny yields inferior masks, while the V-channel based detection is more reliable in our videos. We also test a band-shaped occlusion mask (**Y_band**); however, birds occluded by the subtitle band become undetectable, leading to persistent false negatives.
- **Specular highlight mask (spec_mask, preprocessing).** Disabling **spec_mask** increases false positives, as reflective backgrounds (e.g., water glints and leaf reflections) are more frequently captured by detection boxes.
- **ROI design for camera motion compensation.** Our final design uses a **strip ROI**. We ablate an alternative **strip+corner ROI**; empirically it performs worse than strip-only (Fig. 2(??)), likely because adding corner regions introduces more dynamic background structures and degrades alignment robustness.
- **Frame differencing with KNN (candidate generation).** We introduce a KNN background subtractor to recover candidates when frame differencing produces no/too few candidates (e.g., low-motion birds). We *do not* use an AND fusion. A naive **unconditional union** (always taking KNN foreground together with Diff) can easily cause severe mask expansion and yield near full-frame detections in dynamic backgrounds. Therefore, we use a **gated** strategy: KNN is enabled only when Diff is insufficient; with a relaxed enabling condition, the bird can be detected correctly while avoiding full-frame expansion (Fig. 2(??)). A representative visualization of candidates is provided in Fig. 4a.
- **Background subtraction: KNN vs. MOG2 (candidate generation).** Replacing KNN with **MOG2** degrades performance in our setting, producing less usable foreground masks under illumination changes and dynamic backgrounds (e.g., noisier/less stable segmentation), which hurts subsequent detection and association.

D. LLM Usage Statement

LLM contributed 40% to this project. We used LLM to search for related work and then check the results with GoogleScholar. We also used LLM to search available datasets and papers on motion detection and tracking for the theme of our project. In addition, we used LLM to search for some OpenCV-related methods to help determine the

technical path. We also searched some LaTeX formations with LLM for writing paper, which is much quicker than reading LaTeX guidelines.

E. Contribution Statement

Equal contribution. Jiren Ren and Xi Wang made the same contribution.

- **Jiren Ren:** step preprocessing, step camera motion compensation and step candidate generation.
- **Xi Wang:** step candidate refinement, step tracking and NetTrack finetune.